

CSCE 735 PARALLEL COMPUTING · MODULE 09 · LECTURE NOTES

GPU Optimisation

Getting from working to fast.

MODULE OUTCOMES

- Use shared memory for tiling and reuse.
- Explain and avoid bank conflicts.
- Overlap transfer with computation using streams.
- Identify the limiting resource from a profile.
- Judge when to use a library instead.

1 Shared memory and tiling

TILING TURNS A BANDWIDTH PROBLEM INTO A COMPUTE PROBLEM

Naive matrix multiply reads each element of A and B from global memory $O(n)$ times. Its arithmetic intensity is about 0.25 FLOP per byte (Module 07), which places it firmly on the bandwidth-bound side of the roofline — despite matrix multiply being the canonical arithmetic-heavy kernel.

Tiled matrix multiply loads a $T \times T$ tile of each operand into shared memory once, and every thread in the block then reads it from shared memory, which is roughly twenty times faster and does not consume global bandwidth.

Arithmetic intensity rises with the tile dimension. Each element loaded is used T times, so a 32×32 tile raises the intensity by roughly a factor of 32 — which carries the kernel well past the ridge point.

So exactly the same arithmetic becomes compute-bound. This is the clearest demonstration in the course that arithmetic intensity is something you change rather than something you have, and it is why tiling is the canonical optimisation on every platform.

```

#define T 32
__shared__ float As[T][T], Bs[T][T];
float acc = 0.0f;
for (int t = 0; t < n/T; t++) {
    As[threadIdx.y][threadIdx.x] = A[row*n + t*T + threadIdx.x];
    Bs[threadIdx.y][threadIdx.x] = B[(t*T + threadIdx.y)*n + col];
    __syncthreads();                // tile complete
    for (int k = 0; k < T; k++)
        acc += As[threadIdx.y][k] * Bs[k][threadIdx.x];
    __syncthreads();                // before overwriting next iteration
}
C[row*n + col] = acc;

```

Both `__syncthreads()` calls are necessary. The first ensures the tile is fully loaded before any thread reads it. The second ensures every thread has finished reading before the next iteration overwrites it — and this is the one that is routinely omitted, producing a race that appears only under load and only on some devices.

BANK CONFLICTS SERIALISE SHARED MEMORY

Shared memory is divided into 32 banks, each of which can serve one access per cycle. Successive 32-bit words fall in successive banks.

If the 32 threads of a warp access 32 different banks, all are served in one cycle. Full shared-memory bandwidth.

If they all access the same bank, the accesses serialise — thirty-two cycles instead of one, for the same data. The hardware does this silently and the only symptom is that the kernel is slow.

Column-wise access of a $T \times T$ tile is the classic case. With $T = 32$, elements of a column are 32 words apart, so every thread in a column maps to the same bank — a 32-way conflict on every access. **Declaring the array as $[T][T+1]$ shifts each row by one word**, so a column spans all 32 banks and the conflict disappears entirely. **One character of padding, up to a $32 \times$ difference**, and it is worth measuring to believe.

2 Transfers and streams

PCIe IS THE BOTTLENECK NOBODY BUDGETS FOR

GPU memory bandwidth is on the order of 1000 GB/s. PCIe is on the order of 30 GB/s. A factor of roughly thirty, and the gap has widened over successive generations.

So transferring data to and from the device can cost more than the computation itself, and for a single kernel over a modest dataset it frequently does. A kernel that is ten times faster than the CPU can lose outright once the round trip is counted.

The rule is to keep data resident on the device and run many kernels against it, rather than transferring for each operation. Restructuring a pipeline so the data crosses PCIe once is usually worth more than any kernel optimisation.

And overlap what you genuinely must transfer with computation, using streams. **A GPU result that beats the CPU on kernel time and loses once transfer is included is a common and dishonest claim**, and Project 2's comparison requirement exists partly to prevent it.

```
const int NS = 4;
for (int i = 0; i < nchunks; i++) {
    int k = i % NS;
    cudaMemcpyAsync(d_in+off, h_in+off, bytes, H2D, s[k]);
    kernel<<<grid, block, 0, s[k]>>>(d_in+off, d_out+off, m);
    cudaMemcpyAsync(h_out+off, d_out+off, bytes, D2H, s[k]);
}
// REQUIRES PINNED HOST MEMORY (cudaMallocHost).
```

PINNED MEMORY IS WHAT MAKES ASYNCHRONOUS COPIES ASYNCHRONOUS

Pageable host memory cannot be DMA'd directly, because the operating system may move or swap the pages at any time.

So the driver stages the transfer through an internal pinned buffer — copying from your pageable memory into its pinned buffer, then DMAing from there. **That staging copy is synchronous.**

Which means cudaMemcpyAsync from pageable memory is silently not asynchronous, and a carefully constructed stream pipeline over pageable memory achieves no overlap at all while appearing correct.

Allocate host buffers with cudaMallocHost and the copies become genuine DMA transfers that overlap with computation. **The profiler shows the difference immediately** — the timeline either has overlapping bars or it does not — which is why profiling a stream pipeline is not optional.

3 Finding the limiting resource

Profile symptom	Limiting resource	What helps
High memory throughput, low compute throughput	Memory bandwidth.	Tiling and shared memory (§1); reduce bytes moved; improve layout; use smaller data types.
High compute, low memory	Compute.	A better algorithm; lower precision where acceptable; tensor cores if the operation maps onto them.
Both low	Latency.	The common beginner result — the GPU is waiting rather than working. Raise occupancy, or increase instruction-level parallelism per thread (Module 08 §3).
High warp divergence	Control flow.	Reorganise the data so that warps are uniform (Module 08 §1).
Low achieved bandwidth relative to peak	Coalescing.	Fix the access pattern (Module 08 §2).
Gaps between kernels on the timeline	Launch overhead or transfers.	Fuse small kernels; use streams; check for implicit synchronisation.

Nsight Compute names the limiting resource directly in recent versions, which makes this table a cross-check rather than a diagnostic procedure — but knowing what the categories mean is still necessary to act on the report.

- 1 **Correctness**, verified against a CPU implementation.
- 2 **Coalescing**. Usually the single largest factor, and it is a data layout change rather than a code change (Module 08).
- 3 **Shared memory tiling**, wherever there is data reuse to exploit (§1).
- 4 **Occupancy**, if and only if the profile indicates a latency-bound kernel.
- 5 **Divergence**, if the profile indicates control flow is the limit.
- 6 **Streams and transfer overlap** (§2).
- 7 **Instruction-level tuning** — last, and rarely necessary once the preceding six are done.

Measure after each step. Some interact — tiling changes the shared memory usage and therefore the occupancy — and some steps lose. **This mirrors Module 07's CPU ordering for the same reasons**, and performing the steps out of order wastes most of the effort.

USE THE LIBRARY

cuBLAS, cuFFT, cuDNN, Thrust, and CUB are written by people with access to the architecture documentation, and they are re-tuned for each device generation. Several of their kernels are generated by autotuners exploring configuration spaces no human would enumerate.

A hand-written tiled matrix multiply reaching 60% of cuBLAS is a good result — and it took a week, and it will not be re-tuned when the next architecture ships.

Write your own in order to understand the machine. This is genuinely necessary: you cannot read a profile or judge whether a library call is performing well without knowing what coalescing, occupancy, and divergence mean in practice. That is what this module is for.

And then use theirs. Project 2 requires the comparison precisely because reporting the gap honestly is the exercise — the number is informative, and the instinct to present a hand-written kernel without the comparison is exactly what the requirement exists to prevent.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
NVIDIA — CUDA C++ Best Practices Guide (free)	Coalescing, shared memory, bank conflicts, and streams, with the optimisation ordering of §3.
NVIDIA — Nsight Compute documentation (free)	How to read the profile and what each metric means. The tool names the limiter; this explains it.
Mark Harris — Optimizing Parallel Reduction in CUDA (free)	Seven successive optimisations of one kernel, each measured. The best worked example of §3's methodology that exists, and it is thirty slides.
CUB and Thrust documentation (free)	The library primitives of §3's recommendation, and the source is readable.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Implement naive and tiled matrix multiply. Measure both, compute the arithmetic intensity of each, and place both on the roofline.
- 2 Sweep the tile size from 8 to 32 and plot performance. Explain the shape, including the shared memory limit.
- 3 **Produce a bank conflict** with a column access of a 32×32 tile, measure it, then pad to $[32][33]$ and measure again.

- 4 Omit the second `__syncthreads()` and construct a case where the result is wrong.
- 5 Measure PCIe bandwidth in both directions, with pageable and with pinned host memory.
- 6 Time a kernel with and without the transfer included and report both numbers for the same workload.
- 7 Implement a four-stream pipeline and confirm the overlap in the profiler timeline. Then switch to pageable memory and confirm the overlap disappears.
- 8 **Work through Mark Harris's reduction optimisations**, implementing and measuring each of the seven steps.
- 9 Profile each version with Nsight Compute and record the limiting resource at each stage.
- 10 **Benchmark your tiled matmul against cuBLAS** and report the percentage achieved.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Why does tiling raise arithmetic intensity, and by roughly how much?
- 2 Why are both `__syncthreads()` calls necessary?
- 3 What is a bank conflict, what is the classic case, and what is the fix?
- 4 Give the bandwidth ratio between GPU memory and PCIe, and the rule that follows.
- 5 Why does an asynchronous copy require pinned memory?
- 6 Give six profile symptoms and their limiting resources.
- 7 What does 'both compute and memory low' indicate?
- 8 Give the seven-step GPU optimisation order.
- 9 Why write your own kernel, and why then use the library?